RIDGEWATCH RESCUE INTELLIGENCE PLATFORM

FRONTEND ARCHITECTURE & FILE-BY-FILE CONCEPTUAL SPECIFICATION

AI-Based Multi-Disaster Survivor Detection & Rescue Coordination System
Comprehensive Technical Documentation for Project Review, Mentors, Judges & Maintenance

DOCUMENT METRIC	SYSTEM VALUE	AUDITED SCOPE
System Version	RidgeWatch v2.0 (Production)	Entire Frontend Subsystem
Target Form Factors	Desktop Commander, Mobile Tablet, Wearable Smartwatch	Cross-Platform Responsive
Total Frontend Files	54 Verified Files	54 Files in active codebase
HTML Document Modules	25 HTML Views & Compatibility Aliases	Root & Testing Directories
JavaScript Controllers	28 JavaScript Controllers & Proxies	frontend/js/ Directory
CSS Design System	1 Master Stylesheet (style.css, Light White/Blue)	frontend/css/style.css
Mathematical Integrity	Strict: Total Victims = Rescued + En Route + Pending	Zero Fake Data / Zero Contradictions
Date of Generation	August 21, 2026	Production Build Verification

[DOCUMENT PURPOSE & EXECUTIVE SCOPE] This document provides exhaustive, conceptual, file-by-file technical documentation for the complete frontend architecture of RidgeWatch. It deliberately focuses on operational responsibilities, data flow, user interactions, and visual logic without burdening the reader with raw code syntax.

TABLE OF CONTENTS

1. Executive Summary & Frontend Purpose
2. Frontend Architectural Philosophy & Design Principles
3. System Navigation Architecture & Module Hierarchy
4. Shared Frontend Infrastructure & Centralized Utilities
5. HTML File-by-File Technical & Conceptual Documentation (25 Files)
6. JavaScript File-by-File Technical & Conceptual Documentation (28 Files)
7. CSS Design System & Visual Foundation (style.css)
8. Visual 2D / 3D / 4D Digital Twin Deep Dive
9. Logical AI Decision Center & Explainability Engine Deep Dive
10. Analytics & Operational Intelligence Dashboard Deep Dive
11. Incident Commander & Tactical Map Clustering Engine Deep Dive
12. Site Map, Geodetic Positioning & GPS Engine Deep Dive
13. Hardware & Sensor Station Telemetry Monitoring Deep Dive
14. Admin & Master Configuration Management Subsystem
15. Testing Console & Multi-Disaster Simulation Sandbox
16. End-to-End Operational User Journeys (Workflows A through K)
17. Frontend Data Flow & State Lifecycle Patterns
18. Comprehensive File Relationship & Dependency Matrix
19. 6W Comprehensive Analysis Summary
20. Component Criticality & Dependency Ranking Analysis
21. 'If This File Fails' Systemic Impact & Resilience Analysis
22. Architectural Rationale: Why This Frontend Architecture?
23. The Complete System Frontend Story (End-to-End Narrative)
24. Implementation Status, Compatibility Aliases & Partial Areas
25. Frontend Architecture Summary & Final Conclusions

1. Executive Summary & Frontend Purpose

RIDGEWATCH is an advanced AI-powered multi-disaster survivor detection, triage, and autonomous search-and-rescue coordination system. The frontend serves as the operational command interface, cognitive decision visualization center, and spatial tactical map for emergency disaster responders. It translates complex multi-modal sensor streams (Inclinometer, Rainfall, Thermal Infrared, Optical YOLOv8, Barometric Pressure, Seismic, and Biometric vitals) into clear, actionable intelligence.

Core Operational Questions Answered by the Frontend

1. What is the frontend? A modular, responsive, high-performance web client suite comprising 12 interconnected operational modules, interactive 2D/3D/4D digital twins, tactical spatial maps, and mobile field clients.

2. Why does it exist? In search-and-rescue missions, the 'Golden Hour' dictates survivor mortality. The frontend exists to eliminate cognitive overload, visualize AI survival probability scores (SPS), guide tactical dispatches, and coordinate multi-team field responses.

3. Who uses it? Incident Commanders (overall theater strategy), Search & Rescue Responders (tactical field execution), Medical Triage Officers (survivor health prioritization), Hardware Engineers (sensor station health), and System Evaluators (simulation and training).

4. Where is it used? In Emergency Operations Centers (EOC desktop monitors), forward mobile command vehicles (ruggedized field tablets), and on-foot responder wearables (smartwatch browser view).

5. When is each module used? Dashboard and Commander are active during live operations; Logical is used when auditing AI evidence; Watch is used during active foot search; Analytics is used for performance review; Testing is used for simulation and scenario authoring.

6. How does it work as a complete system? The frontend maintains a decoupled, reactive polling and event-driven architecture that queries standardized REST endpoints, processes JSON operational payloads, manages client-side UI states, and provides zero-latency visual feedback.

> **[ZERO FAKE DATA MANDATE]** Every number, badge, chart, and map entity rendered on the RidgeWatch frontend derives strictly from audited backend simulation models, database records, or live sensor telemetry. The frontend strictly forbids fabricated metrics or hardcoded dummy values.

2. Frontend Architectural Philosophy & Design Principles

The RidgeWatch frontend is engineered around four core architectural pillars: Clean Light White with Blue Visual Ergonomics, Explainable Intelligence Hierarchy, Strict Mathematical Consistency, and High-Contrast Tactical Readability.

Visual Design System: Light White with Blue Palette

Operational emergency control rooms require high legibility under varying ambient lighting conditions. The interface employs a tailored palette consisting of Crisp Canvas White (#FFFFFF), Ice Tinted Backgrounds (#F3F7FD), Deep Navy Typography (#0F2547), Electric Blue Accents (#2563EB), and standardized semantic state badges.

TOKEN NAME	HEX VALUE	SEMANTIC USAGE IN UI
--bg-void	#FFFFFF	Primary canvas, modal containers, and elevated cards
--bg-panel-raised	#F3F7FD	Sidebar navigation, hero banners, and sub-nav bars
--ink	#0F2547	High-contrast headings, primary titles, and numerical metrics
--ink-dim	#64748B	Metadata labels, timestamps, units, and secondary subtitles
--accent-ice	#2563EB	Interactive links, primary buttons, and live route lines
--border	#D3E1F5	Subtle structural borders, table dividers, and input strokes
--status-critical	#EF4444	High-priority survivor triage and emergency hazard alerts
--status-warning	#EA580C	Moderate hazard thresholds and pending dispatch queues
--status-success	#16A34A	Verified survivor rescues and nominal hardware stations

Standardized Data State Indicator System

To prevent operational ambiguity, the frontend enforces a strict universal data state convention across all 12 modules:

STATE INDICATOR	VISUAL REPRESENTATION	OPERATIONAL MEANING
Valid Numerical Zero	0	A verified zero quantity (e.g. 0 active alerts, 0 critical failures)
Missing Telemetry	--	Sensor channel offline, uncalibrated, or awaiting initial packet
Not Applicable	N/A	Metric cannot be mathematically evaluated (e.g. rescue rate when victims = 0)
Awaiting Pipeline	PENDING	Task queued or AI fusion engine computing current step
Device Offline	OFFLINE	Station disconnected or failed heartbeat threshold
Data Stale	STALE	Last received packet timestamp exceeds maximum freshness window (>15s)
Healthy / Nominal	ONLINE / NOMINAL	Subsystem verified healthy and operating within calibrated bounds

3. System Navigation Architecture & Module Hierarchy

The frontend is structured into 12 primary operational modules accessible via the persistent master sidebar, complemented by a secondary testing sub-navigation bar for development and simulation administration.

Master Navigation Tree

RIDGEWATCH SYSTEM NAVIGATION TOPOLOGY: ■■■ ■ Dashboard (index.html) -> Executive Situation Summary & Quick Launcher ■■■ ■ Visual 3D (visual.html) -> 2D Architecture / 3D Digital Twin / 4D Simulation ■■■ ■ Logical (logic.html) -> Level 1–4 AI Decision Center & Step-by-Step Math ■■■ ■■ Commander (commander.html) -> Tactical 2D Clustered Map & Job Dispatch ■■■ ■ Watch (watch.html) -> Mobile / Wearable Field Responder Triage Client ■■■ ■ Alerts (alerts.html) -> Active Emergency Hazard Alert Ledger & Audio Alarms ■■■ ■ Analytics (analytics.html) -> Scope-Filtered Rescue Performance Dashboard ■■■ ■■ Scenarios (scenarios.html) -> Pre-configured Disaster Scenario Catalog ■■■ ■ Victims (victims.html) -> Survivor Directory, SPS Triage & Vital Signs ■■■ ■ Teams (teams.html) -> Rescue Team Deployment Roster & Availability ■■■ ■ Hardware (hardware.html) -> Sensor Diagnostic Gauges & Telemetry Monitor ■■■ ■ Testing Console (testing.html) -> Multi-Disaster Simulation Sandbox ■■■ Scenarios Ledger (testing-scenarios.html) ■■■ Personnel Directory (testing-persons.html) ■■■ Teams Catalog (testing-teams.html) ■■■ Custom Scenario Builder (testing-add-scenario.html) ■■■ Personnel Enrollment (testing-add-person.html) ■■■ Team Formation (testing-add-team.html)

4. Shared Frontend Infrastructure & Centralized Utilities

To guarantee architectural consistency, prevent code duplication, and maintain maintainability, shared frontend responsibilities are decoupled into dedicated shared utilities.

Shared Infrastructure Components

1. js/config.js (API Gateway & Runtime Discovery): Centralizes backend URL detection. Determines whether the web app is served directly from FastAPI, behind a reverse proxy, or on a custom port, providing a unified getApiBase() accessor.

2. js/testing-common.js (Testing Subsystem Framework): Dynamically constructs and injects the secondary sub-navigation bar across all 6 Testing Console views, tracks active page highlights, and manages toast notification popups.

3. css/style.css (Design System Tokens): Declares global CSS variables, resets browser defaults, formats standard card elevation physics, and handles responsive layout collapse for mobile viewports.

> **[SHARED INFRASTRUCTURE FLOW]** CENTRAL CONFIG (js/config.js) -> SHARED LAYOUT (js/testing-common.js) -> GLOBAL THEME (css/style.css) -> 12 OPERATIONAL MODULES.

5. HTML File-by-File Technical Documentation (25 Files)

This section documents every single HTML view and compatibility gateway in the RidgeWatch frontend.

File Specification: index.html

FILE NAME	**index.html**
FILE TYPE	HTML (Core Dashboard View)
PRIMARY PURPOSE	Primary operational dashboard presenting high-level disaster telemetry, scenario quick-start launcher, live mini-map, survivor counts, and active alert banners.
WHAT	The executive dashboard and primary entry portal for operational commanders and search-and-rescue teams.
WHY	Provides immediate situational awareness in the critical first minutes of a disaster without requiring navigation through deep technical sub-pages.
WHO	Incident Commander, Search & Rescue Operators, Field Team Leads.
WHERE	Root level / main landing page of the RidgeWatch web portal.
WHEN	Loaded initially upon accessing the system or clicking the Dashboard link in the master sidebar.
HOW	Renders semantic KPI metric cards, scenario trigger buttons, interactive site map container, and live telemetry badges driven by dashboard.js.
INPUTS	Live operation state from /operations, active victims, sensor telemetry summary, scenario catalog.
OUTPUTS	Rendered operational overview, quick-action dispatch buttons, scenario launch triggers.
UI RESPONSIBILITY	Presents 4 core KPI tiles, scenario selection matrix, live leaflet-styled map canvas, active disaster status banner, and alert notifications.
DEPENDENCIES	css/style.css, js/config.js, js/dashboard.js
IF THIS FILE FAILS	The system loses its primary landing page and executive dashboard, forcing users to access sub-modules directly.

File Specification: visual.html

FILE NAME	**visual.html**
FILE TYPE	HTML (Interactive Multi-Dimensional Visualizer)
PRIMARY PURPOSE	Provides the comprehensive 2D architecture schematic, 3D hardware physical digital twin, and 4D temporal simulation canvas.
WHAT	Hardware and system digital twin visualizer rendering real-time 3D models of drone stations, sensor arrays, and mesh nodes.
WHY	Enables engineers, commanders, and technicians to inspect physical device arrangements, inspect component health in 3D, and track live sensor telemetry particles.
WHO	System Engineers, Hardware Technicians, Incident Commanders.
WHERE	Visual 3D navigation module accessible via sidebar link 'Visual 3D'.
WHEN	Loaded when deep physical spatial analysis, hardware debugging, or 4D time-series progression is required.
HOW	Hosts WebGL canvas containers for Three.js 3D rendering, component inspector sidebar, telemetry overlay HUD, and camera control toolbar.
INPUTS	Hardware telemetry (/hardware/summary, /hardware/telemetry/timeseries), live scenario sensor profiles, user camera gestures.
OUTPUTS	Interactive 3D viewport, 2D architectural schematic, 4D simulation playback, component health inspections.
UI RESPONSIBILITY	Renders Three.js canvas, layer toggle pills (2D / 3D / 4D), hardware component inspector drawer, and environment particle HUD.
DEPENDENCIES	css/style.css, js/config.js, js/visual.js
IF THIS FILE FAILS	The physical digital twin and 3D sensor visualization capabilities become completely unavailable.

File Specification: logic.html

FILE NAME	**logic.html**
FILE TYPE	HTML (AI Decision Center & Explainability)
PRIMARY PURPOSE	Operational AI Decision Center visualizing the Step 0–Step 9 multi-sensor Bayesian fusion, SPS survival scoring, and triage explainability.
WHAT	The cognitive intelligence hub of RidgeWatch showing exactly how and why AI decisions and triage classifications are derived.
WHY	Provides operational commanders with transparent, explainable AI reasoning rather than a black-box recommendation.
WHO	Mission Commander, Triage Officers, System Evaluators, Hackathon Judges.
WHERE	Logical AI Decision module accessed via the sidebar navigation.
WHEN	Used during active operations to inspect evidence weighting, hazard multipliers, and survivor prioritization math.
HOW	Displays Current Decision banner, Evidence Breakdown matrix, Sensor Activation grid, Step 0–Step 9 calculation timeline, and Recommended Actions.
INPUTS	Scenario intelligence (/scenario/intelligence), fusion results (/logic/step-calculations), sensor weights, active alerts.
OUTPUTS	Visual mathematical breakdown, sensor weighting charts, priority triage badges, explainability narratives.
UI RESPONSIBILITY	Renders Level 1 Current Decision, Level 2 Sensor Evidence, Level 3 Calculation Steps, and Level 4 Action Guidance.
DEPENDENCIES	css/style.css, js/config.js, js/logic.js
IF THIS FILE FAILS	Commanders lose all explainability into AI decisions, triage rankings, and multi-sensor fusion math.

File Specification: commander.html

FILE NAME	**commander.html**
FILE TYPE	HTML (Tactical Incident Command Center)
PRIMARY PURPOSE	Tactical mission control dashboard featuring the dynamic 2D operational site map, job assignment workflow, and rescue team tracking.
WHAT	The operational theater command interface for coordinating multi-team rescue missions and assigning jobs.
WHY	Centralizes victim prioritization, job dispatch, team movement tracking, and geodetic site coordination in a single view.
WHO	Incident Commander, Dispatch Coordinator.
WHERE	Commander module accessed via sidebar link 'Commander'.
WHEN	Loaded continuously during active search-and-rescue operations.
HOW	Combines dynamic 2D site map with collision-avoidance label engine, job assignment drawer, staff eligibility table, and live alert feed.
INPUTS	Commander overview API (/commander/overview), live victims, rescue teams, active jobs, member arrivals.
OUTPUTS	Dispatched rescue jobs, real-time map clustering, verified member arrival alerts, tactical status ledger.
UI RESPONSIBILITY	Renders SVG tactical map, interactive legend toggles, job queue drawer, victim cards, and member assignment modals.
DEPENDENCIES	css/style.css, js/config.js, js/commander.js
IF THIS FILE FAILS	Commanders cannot assign rescue jobs, visualize spatial positions on the tactical map, or track team arrivals.

File Specification: watch.html

FILE NAME	**watch.html**
FILE TYPE	HTML (Field Responder Smartwatch / Mobile View)
PRIMARY PURPOSE	Ultra-compact, high-contrast mobile interface optimized for field rescue workers, smartwatch displays, and wearable devices.
WHAT	Mobile field triage notification and assignment client for deployed rescue team members.
WHY	Field responders operating in harsh environments need glanceable victim coordinates, vital signs, and safety alerts without clutter.
WHO	Field Rescue Workers, Paramedics, Search Dogs Handlers.
WHERE	Wearable/Mobile module accessed via sidebar link 'Watch' or mobile browser.
WHEN	Used on ruggedized field tablets, smartphones, and smartwatch browsers during active search operations.
HOW	Renders high-contrast cards showing current assigned victim, GPS coordinates, distance/ETA, and acknowledge buttons.
INPUTS	Active alerts (/alerts), member assignments (/rescue-team/members-status), victim vitals.
OUTPUTS	Field acknowledgment pulses, status transitions (En Route -> Arrived -> Rescued).
UI RESPONSIBILITY	Displays large-format touch buttons, vital signs telemetry, step-by-step navigation compass, and vibration/audio alert cards.
DEPENDENCIES	css/style.css, js/config.js, js/watch.js
IF THIS FILE FAILS	Field personnel lose their dedicated mobile/wearable interface for receiving tactical dispatches.

File Specification: alerts.html

FILE NAME	**alerts.html**
FILE TYPE	HTML (Emergency Hazard Alert Center)
PRIMARY PURPOSE	Dedicated incident alert ledger and real-time audio-visual emergency hazard notification center.
WHAT	Centralized hazard alert ledger tracking active, acknowledged, and resolved multi-hazard warnings.
WHY	Ensures critical life-safety warnings (e.g. secondary landslide, toxic gas cloud, structural collapse) are never missed.
WHO	Incident Commander, Safety Officer, Operations Staff.
WHERE	Alerts module accessed via sidebar link 'Alerts'.
WHEN	Monitored constantly during disasters; pops up high-priority audio-visual banners on critical events.
HOW	Polls active network alerts, groups by severity (CRITICAL, HIGH, WARNING, INFO), and provides acknowledgment actions.
INPUTS	Active alerts ledger (/alerts), network alerts from db, telemetry threshold breaches.
OUTPUTS	Audited alert acknowledgments, severity filter changes, historical alert CSV exports.
UI RESPONSIBILITY	Renders severity filter tabs, active alert cards with countdown timers, audio alarm toggle, and resolution modals.
DEPENDENCIES	css/style.css, js/config.js, js/alerts.js
IF THIS FILE FAILS	The system lacks a dedicated center for auditing, filtering, and acknowledging safety-critical alerts.

File Specification: analytics.html

FILE NAME	**analytics.html**
FILE TYPE	HTML (Rescue Performance & Intelligence Dashboard)
PRIMARY PURPOSE	Comprehensive operational performance dashboard analyzing rescue success rates, response times, hazard matrices, and team effectiveness.
WHAT	The retrospective and operational intelligence center analyzing trends across single-session and historical disaster operations.
WHY	Commanders and stakeholders need mathematically verified insights into rescue speed, survivor survival rates, and station uptime.
WHO	Operations Commander, Disaster Management Directors, Performance Analysts.
WHERE	Analytics module accessed via sidebar link 'Analytics'.
WHEN	Used post-operation or during multi-day disaster campaigns to audit efficiency and sensor reliability.
HOW	Integrates global scope filtering (Live, 24h, 7d, 30d, All-Time), interactive SVG trend charts, scenario donuts, and team rankings.
INPUTS	Analytics summary (/analytics/summary), session records, sensor histories, team arrivals.
OUTPUTS	Performance trend visualizations, response time averages, CSV audit exports, station uptime ratings.
UI RESPONSIBILITY	Renders Hero Operational Picture, 5 Scope KPI cards, SVG trend charts, hazard matrix, sessions table, and system health strip.
DEPENDENCIES	css/style.css, js/config.js, js/analytics.js
IF THIS FILE FAILS	Commanders lose all operational reporting, historical session audits, and performance trend visualizations.

File Specification: scenarios.html

FILE NAME	**scenarios.html**
FILE TYPE	HTML (Disaster Scenario Catalog)
PRIMARY PURPOSE	Master catalog of pre-configured disaster scenarios (Landslide, Earthquake, Flood, Building Collapse, Avalanche, Wildfire, Chemical Leak, etc.).
WHAT	Scenario selection and initiation interface allowing operators to launch multi-hazard simulations.
WHY	Provides a quick, standardized catalog for training drills, capability demonstrations, and operational benchmarking.
WHO	Operators, Evaluators, Instructors.
WHERE	Scenarios module accessed via sidebar link 'Scenarios'.
WHEN	Loaded prior to initiating a new disaster drill or operational simulation.
HOW	Displays scenario cards with geographic origin, active hazard profile, victim count, and one-click launch buttons.
INPUTS	Scenario catalog (/scenarios), scenario sensor profiles.
OUTPUTS	Disaster scenario launch requests (/scenario/start).
UI RESPONSIBILITY	Renders grid of disaster cards with badges indicating disaster category, complexity, and active sensor types.
DEPENDENCIES	css/style.css, js/config.js, js/scenarios.js
IF THIS FILE FAILS	Users cannot visually browse or launch pre-configured disaster scenarios from the catalog.

File Specification: victims.html

FILE NAME	**victims.html**
FILE TYPE	HTML (Survivor Directory & Vital Signs Roster)
PRIMARY PURPOSE	Detailed survivor roster displaying detected victims, triage priorities (HIGH, MEDIUM, LOW), GPS coordinates, and biometric telemetry.
WHAT	Survivor tracking and health intelligence directory.
WHY	Allows triage officers and medical personnel to inspect specific victim vital signs, burial depths, and triage rankings.
WHO	Medical Officers, Triage Personnel, Rescue Coordinators.
WHERE	Victims module accessed via sidebar link 'Victims'.
WHEN	Used whenever detailed medical or vital signs information on a detected survivor is needed.
HOW	Fetches active victim fusion records, renders health status cards, priority filters, and individual victim modal drawers.
INPUTS	Victims endpoint (/victims or /operations/{op_id}/victims), fusion triage scores.
OUTPUTS	Filtered victim lists, medical priority views, coordinate copy actions.
UI RESPONSIBILITY	Renders survivor table with SPS progress bars, heart rate / SpO2 badges, triage class tags, and triage priority filters.
DEPENDENCIES	css/style.css, js/config.js, js/victims.js
IF THIS FILE FAILS	Medical responders lose the dedicated survivor roster view and individual vital sign inspection tools.

File Specification: teams.html

FILE NAME	**teams.html**
FILE TYPE	HTML (Rescue Teams Roster & Deployment Status)
PRIMARY PURPOSE	Rescue team management and deployment directory showing registered units (Alpha, Bravo, Charlie, etc.) and active member assignments.
WHAT	Rescue team operational directory and roster browser.
WHY	Provides visibility into available, en route, and deployed search-and-rescue units across operational sectors.
WHO	Logistics Officers, Incident Commanders.
WHERE	Teams module accessed via sidebar link 'Teams'.
WHEN	Used during deployment planning and team availability tracking.
HOW	Queries registered teams and member states, rendering team status cards, member callsigns, and equipment capabilities.
INPUTS	Registered teams (/admin/teams), live member states (/rescue-team/members-status).
OUTPUTS	Team deployment summaries, availability rosters, direct navigation to Commander dispatch.
UI RESPONSIBILITY	Renders team cards with status badges (Available, Deployed, En Route), member lists, and capability tags.
DEPENDENCIES	css/style.css, js/config.js, js/teams.js
IF THIS FILE FAILS	Operators cannot view team compositions, member assignments, or team availability outside of the commander map.

File Specification: hardware.html

FILE NAME	**hardware.html**
FILE TYPE	HTML (Sensor & Hardware Diagnostic Center)
PRIMARY PURPOSE	Station-by-station hardware telemetry monitor, sensor registry browser, and communication diagnostic dashboard.
WHAT	Hardware and sensor station health intelligence interface.
WHY	Ensures physical sensing stations (Inclinometers, Rain Gauges, Thermal Cameras, Seismic Nodes) are online, calibrated, and transmitting.
WHO	Hardware Technicians, Systems Engineers.
WHERE	Hardware module accessed via sidebar link 'Hardware'.
WHEN	Used to monitor sensor battery levels, signal strength, packet dropouts, and station health.
HOW	Polls hardware registry (/hardware/summary, /hardware/registry), renders live gauge meters, status badges, and sensor details.
INPUTS	Hardware telemetry summaries, sensor battery voltages, calibration statuses.
OUTPUTS	Sensor diagnostic reports, hardware state filters, station restart triggers.
UI RESPONSIBILITY	Renders station health grid, battery meters, live sensor value cards, and diagnostic status pills.
DEPENDENCIES	css/style.css, js/config.js, js/hardware.js
IF THIS FILE FAILS	Technicians lose visibility into sensor node battery levels, communication health, and hardware telemetry.

File Specification: testing.html

FILE NAME	**testing.html**
FILE TYPE	HTML (Interactive Testing Sandbox & Simulator)
PRIMARY PURPOSE	Comprehensive developer and evaluator testing console for injecting synthetic sensor telemetry, simulating dropouts, and advancing scenario cycles.
WHAT	The multi-scenario simulation sandbox and test execution console.
WHY	Enables exhaustive validation of AI logic, threshold alerts, and degradation modes without physical disaster conditions.
WHO	Software Developers, QA Engineers, Hackathon Judges, Evaluators.
WHERE	Testing module accessed via sidebar link 'Testing'.
WHEN	Used during testing, demonstrations, edge-case validation, and automated test runs.
HOW	Hosts interactive simulation controls (Play, Pause, Step, Fast-Forward), telemetry injection sliders, and scenario ledger tables.
INPUTS	Simulation controls, synthetic telemetry payloads, scenario activation requests.
OUTPUTS	Manual telemetry injections, cycle step commands, scenario ledger updates.
UI RESPONSIBILITY	Renders scenario ledger tabs, simulator control toolbar, telemetry slider controls, and real-time response log window.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing.js
IF THIS FILE FAILS	Developers and evaluators cannot interactively inject test sensor values or step through simulation cycles.

File Specification: testing-scenarios.html

FILE NAME	**testing-scenarios.html**
FILE TYPE	HTML (Testing Console - Scenario Ledger)
PRIMARY PURPOSE	Dedicated scenario ledger within the Testing Console for viewing, activating, and toggling disaster scenarios.
WHAT	Testing sub-page managing the active scenario ledger.
WHY	Allows testers to toggle scenarios between Active and Inactive states and launch custom configurations.
WHO	Testers, System Administrators.
WHERE	Testing module sub-navigation.
WHEN	Used to configure which disaster scenarios are offered across the system.
HOW	Displays scenario ledger table with status pills (Active/Inactive), victim counts, and toggle action buttons.
INPUTS	Scenario ledger (/admin/scenarios).
OUTPUTS	Scenario activation/deactivation API calls.
UI RESPONSIBILITY	Renders tabular scenario ledger with search, category filtering, and action buttons.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-scenarios.js
IF THIS FILE FAILS	Testers cannot manage scenario ledger statuses or toggle scenario availability.

File Specification: testing-persons.html

FILE NAME	**testing-persons.html**
FILE TYPE	HTML (Testing Console - Personnel Directory)
PRIMARY PURPOSE	Personnel registry within the Testing Console for viewing certified disaster response personnel, skill matrices, and certifications.
WHAT	Testing sub-page displaying the registered personnel database.
WHY	Enables validation of role eligibility algorithms (e.g. verifying only Avalanche-certified personnel can be assigned to winter missions).
WHO	Testers, Personnel Coordinators.
WHERE	Testing module sub-navigation.
WHEN	Used to review registered staff qualifications and readiness.
HOW	Renders searchable personnel table with badges for certified disaster categories, medical qualifications, and team associations.
INPUTS	Personnel records (/admin/persons).
OUTPUTS	Personnel list rendering, qualification filters, direct link to Add Person form.
UI RESPONSIBILITY	Renders personnel table with skill matrix tags, contact info, and status indicators.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-persons.js
IF THIS FILE FAILS	Testers cannot inspect responder skill matrices or verify role certification records.

File Specification: testing-teams.html

FILE NAME	**testing-teams.html**
FILE TYPE	HTML (Testing Console - Teams Directory)
PRIMARY PURPOSE	Team management interface within the Testing Console for viewing configured rescue units and their assigned member rosters.
WHAT	Testing sub-page managing team definitions and member rosters.
WHY	Allows testers to verify team compositions, member roles, and callsign allocations.
WHO	Testers, Admin Staff.
WHERE	Testing module sub-navigation.
WHEN	Used to inspect team structures and create new tactical units.
HOW	Displays team cards with member rosters, capabilities, base coordinates, and action buttons.
INPUTS	Teams catalog (/admin/teams), members directory.
OUTPUTS	Team overview, deletion/editing triggers, direct link to Add Team form.
UI RESPONSIBILITY	Renders team cards with member avatar pills, base location coordinates, and operational role tags.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-teams.js
IF THIS FILE FAILS	Testers cannot manage team records or inspect member allocations in the testing portal.

File Specification: testing-add-scenario.html

FILE NAME	**testing-add-scenario.html**
FILE TYPE	HTML (Testing Console - Add Custom Scenario Form)
PRIMARY PURPOSE	Form for constructing and registering custom multi-disaster scenarios with custom victim coordinates, hazard types, and site labels.
WHAT	Interactive scenario authoring form.
WHY	Allows creation of arbitrary disaster scenarios for custom drills or localized geography without editing JSON files.
WHO	Instructors, Scenario Authors, Evaluators.
WHERE	Testing module sub-navigation.
WHEN	Used when a new custom scenario needs to be tested or demonstrated.
HOW	Gathers scenario metadata, victim count, site coordinates, and hazard type; submits structured payload to API.
INPUTS	User form input (Scenario Name, Category, Latitude, Longitude, Victim Count, Hazard Severity).
OUTPUTS	Registered custom scenario in database and immediate redirection to scenario ledger.
UI RESPONSIBILITY	Renders validated form inputs, coordinate pickers, hazard dropdowns, and submission feedback.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-add-scenario.js
IF THIS FILE FAILS	Users cannot create new custom scenarios through the web interface.

File Specification: testing-add-person.html

FILE NAME	**testing-add-person.html**
FILE TYPE	HTML (Testing Console - Add Person Form)
PRIMARY PURPOSE	Comprehensive registration form for enrolling rescue responders with specific disaster skill certifications, roles, and callsigns.
WHAT	Personnel enrollment and skill matrix assignment form.
WHY	Ensures new personnel have validated qualifications (e.g. Urban USAR, Canine Search, High-Angle Rope Rescue, Hazmat).
WHO	Personnel Administrators, Training Coordinators.
WHERE	Testing module sub-navigation.
WHEN	Used during initial team onboarding or responder qualification updates.
HOW	Collects personal details, operational role, and multi-select certified disaster categories; validates and posts to /admin/person/create.
INPUTS	User input (Name, Role, Callsign, Certification Checkboxes).
OUTPUTS	Registered responder in personnel database.
UI RESPONSIBILITY	Renders categorized checkbox grid for disaster skills, role selectors, and input validation feedback.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-add-person.js
IF THIS FILE FAILS	New responders cannot be registered with certified skill matrices through the UI.

File Specification: testing-add-team.html

FILE NAME	**testing-add-team.html**
FILE TYPE	HTML (Testing Console - Add Team Form)
PRIMARY PURPOSE	Rescue team assembly form allowing creation of tactical units and assignment of certified, available personnel.
WHAT	Team formation and member assignment form.
WHY	Ensures rescue teams are created with adequate staffing and appropriate base coordinates.
WHO	Incident Commanders, Admin Staff.
WHERE	Testing module sub-navigation.
WHEN	Used to assemble new rescue teams prior to operational deployment.
HOW	Fetches registered unassigned personnel, allows selection of team lead and members, and creates team via /admin/team/create.
INPUTS	Team Name, Team Type, Base Coordinates, Selected Member IDs.
OUTPUTS	Assembled rescue team ready for operational dispatch.
UI RESPONSIBILITY	Renders team metadata fields, member selection picker, and deployment readiness summary.
DEPENDENCIES	css/style.css, js/config.js, js/testing-common.js, js/testing-add-team.js
IF THIS FILE FAILS	New rescue teams cannot be assembled or assigned members via the UI.

File Specification: admin.html

FILE NAME	**admin.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing.html providing backwards-compatible navigation for legacy admin bookmarks.
WHAT	Administrative redirect gateway.
WHY	Maintains seamless routing compatibility for users accessing the legacy /admin.html URL.
WHO	System Users, Legacy Bookmarks.
WHERE	Root level / administrative alias.
WHEN	Triggered when accessing admin.html directly.
HOW	Executes immediate window.location.replace('testing.html') and meta-refresh tag.
INPUTS	URL navigation request.
OUTPUTS	Client redirection to testing.html.
UI RESPONSIBILITY	Displays minimal fallback redirect text if JavaScript is disabled.
DEPENDENCIES	None (Self-contained redirect)
IF THIS FILE FAILS	Direct requests to admin.html would result in a 404 error.

File Specification: admin-scenarios.html

FILE NAME	**admin-scenarios.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-scenarios.html.
WHAT	Scenario ledger alias gateway.
WHY	Ensures legacy admin scenario links route cleanly to testing-scenarios.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-scenarios.html.
HOW	Executes window.location.replace('testing-scenarios.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-scenarios.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-scenarios.html would fail.

File Specification: admin-persons.html

FILE NAME	**admin-persons.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-persons.html.
WHAT	Personnel directory alias gateway.
WHY	Ensures legacy admin personnel links route cleanly to testing-persons.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-persons.html.
HOW	Executes window.location.replace('testing-persons.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-persons.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-persons.html would fail.

File Specification: admin-teams.html

FILE NAME	**admin-teams.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-teams.html.
WHAT	Teams directory alias gateway.
WHY	Ensures legacy admin team links route cleanly to testing-teams.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-teams.html.
HOW	Executes window.location.replace('testing-teams.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-teams.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-teams.html would fail.

File Specification: admin-add-scenario.html

FILE NAME	**admin-add-scenario.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-add-scenario.html.
WHAT	Scenario builder alias gateway.
WHY	Ensures legacy add-scenario links route cleanly to testing-add-scenario.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-add-scenario.html.
HOW	Executes window.location.replace('testing-add-scenario.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-add-scenario.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-add-scenario.html would fail.

File Specification: admin-add-person.html

FILE NAME	**admin-add-person.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-add-person.html.
WHAT	Person registration alias gateway.
WHY	Ensures legacy add-person links route cleanly to testing-add-person.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-add-person.html.
HOW	Executes window.location.replace('testing-add-person.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-add-person.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-add-person.html would fail.

File Specification: admin-add-team.html

FILE NAME	**admin-add-team.html**
FILE TYPE	HTML (Compatibility Alias / Redirect)
PRIMARY PURPOSE	Instant client-side redirect to testing-add-team.html.
WHAT	Team builder alias gateway.
WHY	Ensures legacy add-team links route cleanly to testing-add-team.html.
WHO	System Users.
WHERE	Administrative alias.
WHEN	Triggered when accessing admin-add-team.html.
HOW	Executes window.location.replace('testing-add-team.html').
INPUTS	URL navigation request.
OUTPUTS	Redirection to testing-add-team.html.
UI RESPONSIBILITY	Displays fallback redirect link.
DEPENDENCIES	None
IF THIS FILE FAILS	Legacy bookmarks to admin-add-team.html would fail.

6. JavaScript File-by-File Technical Documentation (28 Files)

This section documents all 28 JavaScript controllers, visual engines, and proxies in frontend/js/.

File Specification: js/config.js

FILE NAME	**js/config.js**
FILE TYPE	JavaScript (Centralized Configuration & API Gateway)
PRIMARY PURPOSE	Provides single-source-of-truth API base URL discovery, automatic port fallback, and runtime configuration helpers.
WHAT	Shared frontend configuration utility.
WHY	Decouples backend host/port bindings from individual page scripts, ensuring seamless switching between local dev, Docker, and production.
WHO	All frontend JavaScript modules.
WHERE	Loaded first in every HTML script bundle.
WHEN	Executed immediately upon page script initialization.
HOW	Exposes the global RidgeWatchConfig object with getApiBase(), getWsBase(), and environment discovery routines.
INPUTS	Browser window.location (protocol, hostname, port), query parameters.
OUTPUTS	Normalized API base URL string (e.g. 'http://127.0.0.1:5000' or relative '').
UI RESPONSIBILITY	None directly; powers all AJAX/fetch network calls across every module.
DEPENDENCIES	None (Pure standard JavaScript)
IF THIS FILE FAILS	Every JavaScript file fails to resolve API endpoints, causing complete network disconnection.

File Specification: js/dashboard.js

FILE NAME	**js/dashboard.js**
FILE TYPE	JavaScript (Dashboard Controller & Polling Engine)
PRIMARY PURPOSE	Drives the main operational dashboard by polling active operation state, rendering KPI counts, updating the mini-map, and managing quick scenario launches.
WHAT	Main dashboard view controller and polling engine.
WHY	Keeps the executive summary updated in real-time with live victim counts, team movements, and active disaster alerts.
WHO	Dashboard page (index.html).
WHERE	Dashboard module.
WHEN	Initializes on index.html load; executes continuous 3-second polling loop.
HOW	Fetches /operations, /victims, /alerts, and /scenarios; updates DOM elements with transition effects; renders Leaflet/SVG mini-map.
INPUTS	Backend REST responses for operations, victims, alerts, scenarios.
OUTPUTS	DOM updates for KPI values, quick-start button states, active scenario badge, and mini-map markers.
UI RESPONSIBILITY	Updates #kpiVictims, #kpiRescued, #kpiActiveAlerts, #scenarioQuickSelect, and #liveMiniMap.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	index.html remains static with placeholder values and no dynamic data updates.

File Specification: js/visual.js

FILE NAME	**js/visual.js**
FILE TYPE	JavaScript (Three.js 3D Engine & Digital Twin Controller)
PRIMARY PURPOSE	Full-featured 3D WebGL rendering engine, 2D architectural schematic renderer, 4D simulation driver, and particle telemetry visualizer.
WHAT	Hardware digital twin 3D engine built on Three.js.
WHY	Provides immersive 3D spatial visualization of hardware stations, sensors, drone paths, and real-time telemetry particles.
WHO	Visual module (visual.html).
WHERE	Visual 3D/4D module.
WHEN	Initializes on visual.html load; runs a 60 FPS requestAnimationFrame render loop.
HOW	Constructs Three.js scene, camera, lights, materials, and procedural 3D hardware meshes; animates particles along sensor data paths; handles mouse raycasting.
INPUTS	Hardware telemetry (/hardware/summary), station registry, user mouse drag/zoom gestures.
OUTPUTS	Interactive 60 FPS WebGL scene, component inspector card updates, telemetry particle streams.
UI RESPONSIBILITY	Renders 3D viewport canvas, 2D schematic overlay, component inspector sidebar, and telemetry HUD.
DEPENDENCIES	js/config.js, external Three.js library (CDN/bundled)
IF THIS FILE FAILS	visual.html cannot render 3D scenes or digital twin hardware models.

File Specification: js/logic.js

FILE NAME	**js/logic.js**
FILE TYPE	JavaScript (AI Decision Center & Explainability Engine)
PRIMARY PURPOSE	Calculates and visualizes the complete Step 0–Step 9 AI reasoning pipeline, Bayesian multi-sensor fusion, and SPS explainability.
WHAT	Frontend AI reasoning engine and step-by-step calculation renderer.
WHY	Translates complex sensor probabilities into transparent, understandable visual steps and actionable rescue guidance.
WHO	Logical page (logic.html).
WHERE	Logical AI Decision module.
WHEN	Initializes on logic.html load; refreshes on live scenario polling cycles.
HOW	Fetches /scenario/intelligence; executes frontend calculation steps; renders Level 1 Decision, Level 2 Evidence, Level 3 Math, and Level 4 Actions.
INPUTS	Scenario intelligence payload, active hazard states, sensor readings.
OUTPUTS	Interactive calculation accordion, sensor weight breakdown, confidence meters, action recommendations.
UI RESPONSIBILITY	Populates #currentDecisionBanner, #evidenceBreakdownGrid, #stepCalculationList, and #recommendedActionsCard.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	logic.html cannot render AI decision steps, calculation proofs, or evidence weighting.

File Specification: js/commander.js

FILE NAME	**js/commander.js**
FILE TYPE	JavaScript (Tactical Command & 2D Map Clustering Engine)
PRIMARY PURPOSE	Powers the Incident Commander hub with dynamic job assignment, 2D spatial map clustering, collision-free label positioning, and member tracking.
WHAT	Tactical incident command workflow and mapping engine.
WHY	Enables seamless job dispatch and spatial tracking without visual clutter from overlapping map labels.
WHO	Commander page (commander.html).
WHERE	Commander module.
WHEN	Initializes on commander.html load; runs a continuous 3-second tactical polling loop.
HOW	Polls /commander/overview; runs clusterOpsJobs spatial clustering; executes bounding-box label collision avoidance; handles job dispatch modals.
INPUTS	Commander overview payload (victims, members, jobs, alerts), user marker clicks, assignment form submissions.
OUTPUTS	Clustered SVG tactical map, dynamic entity counter pills, job dispatch REST calls, arrival notification popovers.
UI RESPONSIBILITY	Renders #commanderSiteMap SVG, #legendCountVictims, #jobQueueDrawer, and #assignmentModal.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	commander.html loses all tactical mapping, job assignment, and team tracking functionality.

File Specification: js/watch.js

FILE NAME	**js/watch.js**
FILE TYPE	JavaScript (Mobile Field Responder Client Controller)
PRIMARY PURPOSE	Manages glanceable smartwatch card updates, mission acknowledgment actions, and emergency audio-vibration alerts.
WHAT	Mobile field responder client script.
WHY	Provides responsive, battery-efficient updates for field rescue workers on mobile and wearable form factors.
WHO	Watch page (watch.html).
WHERE	Watch / Mobile module.
WHEN	Initializes on watch.html load; polls active assignments every 4 seconds.
HOW	Fetches responder mission status, updates large-format victim distance card, and dispatches acknowledgment pulses.
INPUTS	Member status API (/rescue-team/members-status), active alerts.
OUTPUTS	Status transition requests (En Route -> Arrived), DOM card updates, audio alarm triggers.
UI RESPONSIBILITY	Updates #watchVictimCard, #watchVitalsGrid, #watchActionBtn, and #watchAlertBanner.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	watch.html remains static without real-time dispatch updates or acknowledgment buttons.

File Specification: js/alerts.js

FILE NAME	**js/alerts.js**
FILE TYPE	JavaScript (Active Hazard Alert Lifecycle Controller)
PRIMARY PURPOSE	Controls active emergency alert polling, severity filtering, audio alert synthesis, and alert acknowledgment workflows.
WHAT	Emergency alert center view controller.
WHY	Ensures real-time hazard warnings are audibly signaled, visually prioritized, and recorded in compliance with safety protocols.
WHO	Alerts page (alerts.html).
WHERE	Alerts module.
WHEN	Initializes on alerts.html load; polls active alerts ledger every 3 seconds.
HOW	Fetches /alerts; sorts by severity (CRITICAL first); synthesizes Web Audio API alarm pulses on new critical alerts; manages filter tabs.
INPUTS	Alerts endpoint (/alerts), user severity tab clicks, acknowledge button clicks.
OUTPUTS	Updated alert cards, audio alarms, POST /alerts/{id}/ack acknowledgment calls.
UI RESPONSIBILITY	Renders #alertsGrid, #alertFilterTabs, #audioAlarmToggle, and #criticalAlertCounter.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	alerts.html cannot poll alerts, filter by severity, or trigger audio warning alarms.

File Specification: js/analytics.js

FILE NAME	**js/analytics.js**
FILE TYPE	JavaScript (Analytics Engine & Mathematical Consistency Controller)
PRIMARY PURPOSE	Aggregates operational metrics across selectable timeframes (Live, 24h, 7d, 30d, All-Time), renders interactive SVG charts, and enforces mathematical consistency.
WHAT	Operational performance analytics aggregator and visualization engine.
WHY	Provides verified operational metrics (Total Victims = Rescued + En Route + Pending; Rescue Rate = Rescued / Victims) without fake data.
WHO	Analytics page (analytics.html).
WHERE	Analytics module.
WHEN	Initializes on analytics.html load; re-computes all metrics on scope or scenario filter change.
HOW	Fetches /analytics/summary, /scenario/intelligence, /hazards/landslide/history; applies scope filter; draws SVG bars, lines, and donuts.
INPUTS	Analytics summary data, live operation state, sensor history telemetry, user scope dropdown selection.
OUTPUTS	Calculated KPI values, SVG rescue trend chart, scenario donut chart, victim status donut, team performance cards.
UI RESPONSIBILITY	Populates #operationalPictureCard, #kpiRescueRate, #performanceChart, #hazardDonutSvg, #victimStatusDonutSvg, and #sessionsTable.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	analytics.html displays no charts, calculations, or session tables.

File Specification: js/scenarios.js

FILE NAME	**js/scenarios.js**
FILE TYPE	JavaScript (Scenario Catalog Controller)
PRIMARY PURPOSE	Fetches pre-configured disaster scenarios, renders catalog cards with hazard tags, and handles one-click scenario launch triggers.
WHAT	Scenario catalog view controller.
WHY	Enables users to visually explore and launch disaster drills from the scenario library.
WHO	Scenarios page (scenarios.html).
WHERE	Scenarios module.
WHEN	Initializes on scenarios.html load.
HOW	Fetches /scenarios; constructs disaster cards with complexity ratings; binds click handlers to trigger POST /scenario/start.
INPUTS	Scenario catalog REST response, user launch button clicks.
OUTPUTS	Rendered scenario cards, scenario start API calls, navigation to Dashboard.
UI RESPONSIBILITY	Populates #scenariosGrid with scenario cards and launch state feedback.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	scenarios.html cannot fetch or display the scenario library.

File Specification: js/victims.js

FILE NAME	**js/victims.js**
FILE TYPE	JavaScript (Survivor Directory & Triage Controller)
PRIMARY PURPOSE	Manages survivor roster display, triage priority filtering, vital signs telemetry badges, and individual survivor modal views.
WHAT	Survivor directory view controller.
WHY	Provides medical and triage staff with live filtered views of all detected survivors and their health parameters.
WHO	Victims page (victims.html).
WHERE	Victims module.
WHEN	Initializes on victims.html load; polls active survivors every 4 seconds.
HOW	Fetches /victims; filters by triage priority (High, Medium, Low); renders health status pills and coordinate detail drawers.
INPUTS	Victims fusion payload, user triage filter clicks.
OUTPUTS	Filtered victim cards and tables, vital signs progress indicators.
UI RESPONSIBILITY	Populates #victimsTableBody, #triageFilterPills, and #victimDetailModal.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	victims.html cannot display detected survivors or vital signs telemetry.

File Specification: js/teams.js

FILE NAME	**js/teams.js**
FILE TYPE	JavaScript (Rescue Teams Roster Controller)
PRIMARY PURPOSE	Polls registered rescue teams, renders team capability cards, displays member availability, and links to the Commander map.
WHAT	Teams roster view controller.
WHY	Provides a clean overview of all rescue teams, callsigns, and deployment readiness.
WHO	Teams page (teams.html).
WHERE	Teams module.
WHEN	Initializes on teams.html load; polls team states every 5 seconds.
HOW	Fetches /admin/teams and /rescue-team/members-status; renders team summary cards with member count badges.
INPUTS	Teams catalog, member status records.
OUTPUTS	Rendered team cards, deployment status indicators.
UI RESPONSIBILITY	Populates #teamsContainer with structured team cards and member callsign pills.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	teams.html cannot display rescue team cards or member statuses.

File Specification: js/hardware.js

FILE NAME	**js/hardware.js**
FILE TYPE	JavaScript (Hardware Registry & Sensor Diagnostic Controller)
PRIMARY PURPOSE	Monitors sensing stations in real time, renders battery gauge meters, tracks communication dropout states, and displays raw sensor telemetry.
WHAT	Hardware and sensor station diagnostic controller.
WHY	Enables engineers to monitor physical sensing nodes, diagnose low battery states, and detect dropped sensor packets.
WHO	Hardware page (hardware.html).
WHERE	Hardware module.
WHEN	Initializes on hardware.html load; executes continuous 3-second diagnostic polling loop.
HOW	Fetches /hardware/summary and /hardware/registry; calculates battery percentages; renders SVG gauges and hardware health cards.
INPUTS	Hardware telemetry records, sensor battery voltages, calibration metadata.
OUTPUTS	DOM updates for hardware station cards, battery meters, sensor status pills.
UI RESPONSIBILITY	Populates #hardwareGrid, #stationHealthGauges, and #rawTelemetryInspector.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	hardware.html displays no hardware cards or battery diagnostic gauges.

File Specification: js/testing-common.js

FILE NAME	**js/testing-common.js**
FILE TYPE	JavaScript (Testing & Admin Shared Layout Framework)
PRIMARY PURPOSE	Injects the unified navigation header, active tab highlighters, common modals, and notification toast handlers across all Testing Console sub-pages.
WHAT	Shared layout and navigation framework for the Testing Console.
WHY	Eliminates duplicate navigation HTML across the 7 Testing/Admin sub-pages and enforces consistent header styling.
WHO	All Testing Console pages (testing*.html).
WHERE	Shared across Testing & Admin modules.
WHEN	Loaded first on every testing-*.html page before page-specific scripts.
HOW	Dynamically constructs and inserts the secondary sub-nav bar, active tab indicators, and toast notification container.
INPUTS	Current page filename (window.location.pathname).
OUTPUTS	Rendered shared header sub-nav, active tab highlight, toast notifications.
UI RESPONSIBILITY	Renders .testing-nav-bar, #testingToastContainer, and common modal backdrops.
DEPENDENCIES	js/config.js
IF THIS FILE FAILS	Testing and Admin sub-pages lose their unified sub-navigation bar and toast notification system.

File Specification: js/testing.js

FILE NAME	**js/testing.js**
FILE TYPE	JavaScript (Interactive Simulation Driver & Sandbox Controller)
PRIMARY PURPOSE	Drives the main Testing Console sandbox, handles simulation cycle step controls, executes synthetic sensor injections, and displays live simulation logs.
WHAT	Testing console simulation controller and synthetic data injector.
WHY	Allows testers to manipulate simulation speed, advance cycles, trigger degradation modes, and test edge-case sensor values.
WHO	Testing page (testing.html).
WHERE	Testing module.
WHEN	Initializes on testing.html load.
HOW	Binds simulation control buttons (Step, Play, Reset); posts telemetry payloads to /testing/inject-reading; displays streaming simulation event logs.
INPUTS	User slider adjustments, step button clicks, scenario selector.
OUTPUTS	Telemetry injection API calls, cycle step requests, formatted log output.
UI RESPONSIBILITY	Controls #simControlBar, #telemetrySliderInputs, #activeScenarioInfo, and #simLogViewer.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing.html cannot advance simulation steps or inject test telemetry.

File Specification: js/testing-scenarios.js

FILE NAME	**js/testing-scenarios.js**
FILE TYPE	JavaScript (Testing Scenario Ledger Controller)
PRIMARY PURPOSE	Manages the scenario ledger table in the Testing Console, allowing testers to toggle scenarios Active/Inactive and launch fresh test runs.
WHAT	Scenario ledger controller.
WHY	Provides administrative control over which scenarios are enabled in the database ledger.
WHO	testing-scenarios.html.
WHERE	Testing module.
WHEN	Initializes on testing-scenarios.html load.
HOW	Fetches /admin/scenarios; renders ledger rows with toggle switches; calls POST /admin/scenario/{id}/toggle on status change.
INPUTS	Scenario ledger data, toggle switch clicks.
OUTPUTS	Updated scenario ledger states, scenario activation API calls.
UI RESPONSIBILITY	Populates #scenariosTableBody with status switches and victim count badges.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-scenarios.html cannot display or toggle scenario ledger statuses.

File Specification: js/testing-persons.js

FILE NAME	**js/testing-persons.js**
FILE TYPE	JavaScript (Testing Personnel Directory Controller)
PRIMARY PURPOSE	Manages the responder personnel table in the Testing Console, providing search, role filtering, and skill certification inspection.
WHAT	Personnel directory controller.
WHY	Allows inspection of registered personnel, disaster certifications, and contact information.
WHO	testing-persons.html.
WHERE	Testing module.
WHEN	Initializes on testing-persons.html load.
HOW	Fetches /admin/persons; provides text search and skill filtering; renders formatted personnel table.
INPUTS	Personnel records, search input, role filter dropdown.
OUTPUTS	Filtered personnel table, delete person API calls.
UI RESPONSIBILITY	Populates #personsTableBody with responder records and skill certification tags.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-persons.html cannot fetch or display registered responder personnel.

File Specification: js/testing-teams.js

FILE NAME	**js/testing-teams.js**
FILE TYPE	JavaScript (Testing Teams Management Controller)
PRIMARY PURPOSE	Controls the teams table and member cards in the Testing Console, allowing team inspection and deletion.
WHAT	Teams management controller.
WHY	Allows testers and administrators to manage team configurations and view member allocations.
WHO	testing-teams.html.
WHERE	Testing module.
WHEN	Initializes on testing-teams.html load.
HOW	Fetches /admin/teams; renders structured team cards with member rosters; handles team deletion requests.
INPUTS	Teams records, delete button clicks.
OUTPUTS	Rendered team list, DELETE /admin/team/{id} calls.
UI RESPONSIBILITY	Populates #teamsTableBody with team rows, member counts, and action buttons.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-teams.html cannot display or delete configured rescue teams.

File Specification: js/testing-add-scenario.js

FILE NAME	**js/testing-add-scenario.js**
FILE TYPE	JavaScript (Custom Scenario Form Handler)
PRIMARY PURPOSE	Validates custom scenario builder inputs, generates synthetic victim coordinates around the site origin, and posts to /admin/scenario/create.
WHAT	Custom scenario builder controller.
WHY	Enables creation of custom multi-hazard scenarios with validated geographical and victim parameters.
WHO	testing-add-scenario.html.
WHERE	Testing module.
WHEN	Initializes on testing-add-scenario.html form submission.
HOW	Validates form fields, generates procedural victim coordinates, submits payload to API, and navigates to scenario ledger on success.
INPUTS	Form field values (Scenario ID, Name, Coordinates, Victim Count, Hazard Type).
OUTPUTS	Validated scenario creation API request, user validation alerts.
UI RESPONSIBILITY	Handles form validation states, error highlights, and submission loading spinners.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-add-scenario.html cannot validate or submit custom scenario creation requests.

File Specification: js/testing-add-person.js

FILE NAME	**js/testing-add-person.js**
FILE TYPE	JavaScript (Personnel Enrollment Form Handler)
PRIMARY PURPOSE	Validates responder enrollment data, collects multi-disaster skill certifications, and submits to /admin/person/create.
WHAT	Personnel enrollment form controller.
WHY	Ensures all registered responders have valid names, roles, and appropriate disaster category certifications.
WHO	testing-add-person.html.
WHERE	Testing module.
WHEN	Initializes on testing-add-person.html form submission.
HOW	Collects checked disaster categories; validates callsign and role; posts JSON payload to /admin/person/create; redirects to personnel directory.
INPUTS	Form inputs (Name, Role, Callsign, Certification Checkboxes).
OUTPUTS	Person creation API request, error/success toast notifications.
UI RESPONSIBILITY	Controls form validation states and skill category checkbox groupings.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-add-person.html cannot validate or register new personnel.

File Specification: js/testing-add-team.js

FILE NAME	**js/testing-add-team.js**
FILE TYPE	JavaScript (Team Creation Form Handler)
PRIMARY PURPOSE	Fetches available personnel, validates team metadata, allows member selection, and posts new team definitions to /admin/team/create.
WHAT	Team creation form controller.
WHY	Enables assembling new tactical units by associating available personnel with team metadata.
WHO	testing-add-team.html.
WHERE	Testing module.
WHEN	Initializes on testing-add-team.html load and form submission.
HOW	Queries /admin/persons to populate member picker; collects team parameters; posts to /admin/team/create; redirects to teams directory.
INPUTS	Form inputs (Team Name, Team Type, Base Coordinates, Selected Member IDs).
OUTPUTS	Team creation API request, member selection updates.
UI RESPONSIBILITY	Populates member selection checkboxes, validates minimum staffing, and handles submission.
DEPENDENCIES	js/config.js, js/testing-common.js
IF THIS FILE FAILS	testing-add-team.html cannot fetch available members or create rescue teams.

File Specification: js/admin.js

FILE NAME	**js/admin.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Redirects legacy admin script invocations to testing.js.
WHAT	Legacy admin controller proxy.
WHY	Maintains backwards compatibility for legacy admin scripts.
WHO	Legacy admin modules.
WHERE	Admin compatibility layer.
WHEN	Loaded if legacy admin.html is requested.
HOW	Delegates execution to testing.js workflows.
INPUTS	None
OUTPUTS	Compatibility routing.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing.js
IF THIS FILE FAILS	Legacy script imports might fail if explicitly referenced.

File Specification: js/admin-common.js

FILE NAME	**js/admin-common.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy admin common layout routines to testing-common.js.
WHAT	Legacy layout proxy.
WHY	Maintains backwards compatibility for admin sub-navigation.
WHO	Legacy admin sub-pages.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin HTML pages.
HOW	Executes testing-common.js initialization.
INPUTS	None
OUTPUTS	Navigation rendering.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-common.js
IF THIS FILE FAILS	Legacy admin pages lose sub-nav if accessed directly without redirect.

File Specification: js/admin-scenarios.js

FILE NAME	**js/admin-scenarios.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy admin scenario ledger calls to testing-scenarios.js.
WHAT	Scenario ledger proxy.
WHY	Ensures legacy script references execute testing-scenarios logic.
WHO	admin-scenarios.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-scenarios.html.
HOW	Delegates to testing-scenarios.js.
INPUTS	None
OUTPUTS	Scenario ledger updates.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-scenarios.js
IF THIS FILE FAILS	Legacy script reference fails.

File Specification: js/admin-persons.js

FILE NAME	**js/admin-persons.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy admin personnel directory calls to testing-persons.js.
WHAT	Personnel directory proxy.
WHY	Ensures legacy script references execute testing-persons logic.
WHO	admin-persons.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-persons.html.
HOW	Delegates to testing-persons.js.
INPUTS	None
OUTPUTS	Personnel directory updates.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-persons.js
IF THIS FILE FAILS	Legacy script reference fails.

File Specification: js/admin-teams.js

FILE NAME	**js/admin-teams.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy admin teams calls to testing-teams.js.
WHAT	Teams management proxy.
WHY	Ensures legacy script references execute testing-teams logic.
WHO	admin-teams.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-teams.html.
HOW	Delegates to testing-teams.js.
INPUTS	None
OUTPUTS	Teams directory updates.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-teams.js
IF THIS FILE FAILS	Legacy script reference fails.

File Specification: js/admin-add-scenario.js

FILE NAME	**js/admin-add-scenario.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy add-scenario form calls to testing-add-scenario.js.
WHAT	Scenario creation proxy.
WHY	Ensures legacy script references execute testing-add-scenario logic.
WHO	admin-add-scenario.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-add-scenario.html.
HOW	Delegates to testing-add-scenario.js.
INPUTS	None
OUTPUTS	Scenario creation requests.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-add-scenario.js
IF THIS FILE FAILS	Legacy script reference fails.

File Specification: js/admin-add-person.js

FILE NAME	**js/admin-add-person.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy add-person form calls to testing-add-person.js.
WHAT	Person registration proxy.
WHY	Ensures legacy script references execute testing-add-person logic.
WHO	admin-add-person.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-add-person.html.
HOW	Delegates to testing-add-person.js.
INPUTS	None
OUTPUTS	Person registration requests.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-add-person.js
IF THIS FILE FAILS	Legacy script reference fails.

File Specification: js/admin-add-team.js

FILE NAME	**js/admin-add-team.js**
FILE TYPE	JavaScript (Compatibility Proxy)
PRIMARY PURPOSE	Proxies legacy add-team form calls to testing-add-team.js.
WHAT	Team creation proxy.
WHY	Ensures legacy script references execute testing-add-team logic.
WHO	admin-add-team.html.
WHERE	Admin compatibility layer.
WHEN	Loaded by legacy admin-add-team.html.
HOW	Delegates to testing-add-team.js.
INPUTS	None
OUTPUTS	Team creation requests.
UI RESPONSIBILITY	None directly.
DEPENDENCIES	js/testing-add-team.js
IF THIS FILE FAILS	Legacy script reference fails.

7. CSS Design System & Visual Foundation (style.css)

frontend/css/style.css contains over 8,900 lines of structured vanilla CSS with zero third-party framework bloat. It provides a modular, highly performant styling engine tailored specifically for real-time telemetry dashboards.

Core CSS Architecture Subsystems

• **Design Tokens & CSS Custom Properties:** Defines foundational color tokens, spacing scales (4px, 8px, 12px, 16px, 24px, 32px), border radii (6px, 10px, 14px, 9999px), and font families ('Space Grotesk' for display headers, 'JetBrains Mono' for numbers/telemetry).

• **Responsive Shell & Navigation:** Implements the fixed .sidebar container (240px width on desktop) paired with the flexible .main-content viewport and sticky topbars.

• **Component Card System (.panel, .stat-card, .analytics-kpi-card):** Provides standardized white elevated cards with subtle border strokes (#D3E1F5), soft ambient drop shadows, and responsive grid layouts.

• **Spatial Tactical Map Styling:** Styles SVG map boundaries, clustered team pins (.cmd-cluster-pin), survivor triage markers (.victim-pin-high, .victim-pin-med), and dynamic collision labels.

• **Responsive Media Breakpoints:** Adapts layouts smoothly at 1440px (wide desktop), 1200px (standard laptop), 960px (tablet landscape), 768px (tablet portrait), and 600px (mobile phones/wearables).

8. Visual 2D / 3D / 4D Digital Twin Deep Dive

The Visual module (visual.html and visual.js) provides multi-dimensional digital twin representations of the disaster deployment site.

Dimensional Hierarchy

• **2D Architectural Schematic View:** A clean schematic representation illustrating logical relationships between sensing nodes, gateway routers, drone relays, and the central command server.

• **3D Physical Digital Twin:** Interactive Three.js WebGL spatial model rendering exact hardware enclosures, mast structures, solar panels, sensor probes, and drone flight corridors.

• **4D Temporal Telemetry & Simulation Playback:** Integrates the time dimension, streaming real-time sensor telemetry particles through the 3D model, animating thermal heat plumes, and tracking ground tilt displacement over time.

Interactive Visual Controls & Features

• **Component Inspector Drawer:** Clicking any 3D station opens a slide-over panel displaying real-time battery voltage, packet transmission rate, temperature, and calibration status.

• **Camera Navigation Toolbar:** Provides orbit controls, smooth zoom, pan, preset camera angles (Top-Down Overview, Station Close-Up, Drone Perspective), and full-screen toggle.

9. Logical AI Decision Center & Explainability Engine Deep Dive

The Logical module (logic.html and logic.js) transforms raw sensor probabilities into an operational AI Decision Center structured across 4 distinct cognitive levels.

The 5-Stage Cognitive Decision Pipeline

EVIDENCE (Multi-Sensor Telemetry) ↓ FUSION (Bayesian Probability Weighting) ↓ DECISION (Triage Classification & Priority Score) ↓ EXPLANATION (Primary Driver & Hazard Multiplier Narrative) ↓ ACTION (Recommended Tactical Rescue Dispatch Guidance)

The 4 Information Levels in UI

• **LEVEL 1 — CURRENT DECISION:** Top-level summary card displaying active disaster classification, current risk severity, top survivor triage priority, and confidence rating.

• **LEVEL 2 — SENSOR EVIDENCE:** Visual breakdown separating Primary Sensors (e.g. Inclinometer, Rainfall for Landslide) from Supporting and Inactive sensors.

• **LEVEL 3 — STEP-BY-STEP CALCULATION PROOFS:** Expandable calculation accordion detailing Step 0 (Raw Normalization) through Step 9 (Survival Probability Score derivation).

• **LEVEL 4 — ACTION GUIDANCE:** Explicit operational directives indicating required team specializations, evacuation corridors, and equipment requirements.

10. Analytics & Operational Intelligence Dashboard Deep Dive

The Analytics module (analytics.html and analytics.js) delivers retrospective and real-time operational intelligence.

The Operational Data Hierarchy

DATA (Sensor Telemetry & Timestamps) ↓ MEANING (Calculated Rescue Rates & Response Averages) ↓ IMPACT (Hazard Severity & Station Health) ↓ ACTION (Resource Reallocation & Operational Improvements)

Global Scope Control & Data Integrity

The dashboard features a global scope filter bar with 5 selectable modes: 1. Current Session (Live active operation in memory) 2. Last 24 Hours (Audited SQLite sessions within 24h) 3. Last 7 Days (Historical sessions within 7 days) 4. Last 30 Days (Historical sessions within 30 days) 5. All Time (Entire database audit history) All cards, SVG charts, donuts, and tables update simultaneously to reflect strictly the selected scope, preventing data mixing.

> **[MATHEMATICAL CONSISTENCY RULE]** Total Victims = Rescued + En Route + Pending. Rescue Rate = (Total Rescued / Total Victims) * 100. If Total Victims = 0, the system displays 'N/A · Waiting for sufficient rescue data' rather than a misleading 0.0%.

11. Incident Commander & Tactical Map Clustering Engine Deep Dive

The Commander module (commander.html and commander.js) serves as the primary tactical theater interface.

The Tactical Command Workflow

SCENARIO ACTIVATED ↓ ELIGIBLE STAFF FILTERED (Disaster Certification Check) ↓ TACTICAL TEAM FORMED ↓ SURVIVOR JOB GENERATED ↓ COMMANDER ASSIGNMENT ↓ STAFF DISPATCH (En Route) ↓ ARRIVAL & RESCUE CONFIRMATION (Arrived / Rescued) ↓ COMMANDER MONITORING

2D Tactical Map Engine Features

• **Proximity Job Clustering (clusterOpsJobs):** Automatically groups nearby tactical missions occurring within 80 meters into clean cluster badges to eliminate map clutter.

• **Collision-Free Label Positioning (getSmartLabelAttrs):** A 2D bounding-box geometric collision engine that calculates 8 candidate compass offsets (N, NE, E, SE, S, SW, W, NW) to prevent text overlapping.

• **Operational Theater Boundary Enforcement:** Filters out stale GPS coordinates from unrelated historical sessions (>3.5 km away) to maintain focus strictly on the active incident perimeter.

12. Site Map, Geodetic Positioning & GPS Engine Deep Dive

The spatial engine converts raw WGS-84 GPS coordinates (Latitude, Longitude, Altitude) into pixel-accurate local Cartesian SVG and Leaflet coordinates.

Coordinate Transformations & Navigation Controls

• **Geographic Origin Anchor:** Each scenario establishes a site_origin anchor (e.g. 30.5524° N, 79.1538° E for Himalayan Landslide). All sensor and victim offsets are geodetically computed relative to this datum.

• **Interactive Pan & Zoom Engine:** Supports continuous mouse-wheel zoom, drag-to-pan, and double-click deep inspection.

• **Auto Fit-All Theater Extents:** Calculates the dynamic bounding box encompassing all victims, teams, and sensors, auto-centering the viewport with a single click.

13. Hardware & Sensor Station Telemetry Monitoring Deep Dive

The Hardware module (hardware.html and hardware.js) provides physical layer observability.

Telemetry Diagnostic Flow

PHYSICAL HARDWARE NODE ↓ STATION REGISTRY & BATTERY STATUS ↓ TELEMETRY VOLTAGE & DROPOUT MONITOR ↓ DIAGNOSTIC HEALTH RATING (Nominal / Degraded / Offline) ↓ SCENARIO RELEVANCE CLASSIFICATION

Monitored Hardware Modalities

1. Inclinometer & Biaxial Tiltmeters (Slope stability angles in degrees) 2. Tipping-Bucket Rain Gauges (Precipitation intensity in mm/h and cumulative mm) 3. Ultrasonic Water Level Transducers (Flood reservoir depths in cm) 4. FLIR LWIR Long-Wave Thermal Cameras (Sub-surface human heat signatures) 5. Optical YOLOv8 Video Feeds (Visible survivor detection and confidence) 6. Barometric Pressure Transducers (Atmospheric depression in hPa) 7. Gas & Smoke Multi-Sensors (Toxic gas concentrations and smoke density %)

14. Admin & Master Configuration Management Subsystem

The Admin subsystem provides administrative authority over master records, team formation, personnel registries, and scenario availability.

Core Administrative Responsibilities

• **Personnel Registry Management:** Enroll responders, certify disaster specializations (Canine, Dive, High-Angle Rope, Hazmat, USAR), and update operational statuses.

• **Tactical Team Assembly:** Create rescue teams, designate team leads, assign certified personnel, and define home base coordinates.

• **Scenario Ledger Control:** Toggle disaster scenarios Active/Inactive to control which options appear in operational dispatch dropdowns.

15. Testing Console & Multi-Disaster Simulation Sandbox

The Testing Console (testing.html, testing-*.html, and js/testing*.js) is an exhaustive simulation sandbox for developers, QA engineers, instructors, and judges.

Simulation Sandbox Capabilities

• **Step-by-Step Cycle Stepping:** Advance simulation cycles individually or at accelerated rates (1x, 2x, 5x, 10x) to observe Bayesian probability convergence.

• **Synthetic Sensor Telemetry Injection:** Manually override sensor readings using interactive sliders to simulate extreme disaster threshold breaches (e.g. 50 mm/h rain, 12° tilt).

• **Sensor Degradation & Dropout Testing:** Simulate hardware failures, low battery dropouts, and packet loss to evaluate multi-sensor fusion resilience.

16. End-to-End Operational User Journeys (Workflows A through K)

This section traces the 11 comprehensive end-to-end user journeys supported across the RidgeWatch frontend architecture.

Workflow A: Admin Creates a Custom Disaster Scenario

1. Admin opens testing-add-scenario.html. 2. Enters scenario title, disaster category, geographic coordinates, and victim count. 3. Submits form; testing-add-scenario.js validates inputs and calls POST /admin/scenario/create. 4. System registers scenario in SQLite and redirects to Scenarios Ledger (testing-scenarios.html). 5. New scenario becomes immediately available across Dashboard and Scenarios catalog.

Workflow B: Admin Enrolls a Certified Rescue Responder

1. Admin opens testing-add-person.html. 2. Enters name, callsign, and selects certified disaster specializations (e.g. Avalanche, Landslide). 3. Submits form; testing-add-person.js validates and calls POST /admin/person/create. 4. Person is registered and appears in Personnel Directory (testing-persons.html).

Workflow C: Admin Assembles a Tactical Rescue Team

1. Admin opens testing-add-team.html. 2. Enters team name (e.g. Team Delta), selects type, base coordinates, and checks certified unassigned personnel. 3. Submits form; testing-add-team.js calls POST /admin/team/create. 4. Team is formed and appears in Teams Directory (teams.html and testing-teams.html).

Workflow D: Commander Launches an Operational Disaster Scenario

1. Commander opens Dashboard (index.html) or Scenarios Catalog (scenarios.html). 2. Clicks 'Launch Scenario' on Himalayan Landslide Crisis. 3. Frontend calls POST /scenario/start; simulator initializes sensor stations and victim positions. 4. Dashboard updates active disaster banner and starts live telemetry streaming.

Workflow E: Sensing Platform Produces Live Telemetry

1. Sensor simulator advances cycles; Inclinometers detect 9.5° tilt, rain gauges report 32 mm/h rain. 2. Hardware station (hardware.html) updates live gauges and turns warning badges orange. 3. Visual 3D (visual.html) streams particle telemetry pulses along data cables.

Workflow F: AI Decision Center Derives Explainable Triage Decision

1. Commander opens Logical AI Decision Center (logic.html). 2. Level 1 shows Current Risk: CRITICAL (78/100). 3. Level 2 highlights Soil Moisture and Tilt as Primary Modalities. 4. Level 3 displays Step 0–Step 9 mathematical calculations deriving Survival Probability Score (SPS 88.4%). 5. Level 4 recommends immediate dispatch of Search & Rescue Alpha.

Workflow G: Emergency Hazard Alert Triggered & Audited

1. Slope displacement breaches safety threshold; backend dispatches CRITICAL network alert. 2. Topbar notification dot illuminates; Alerts page (alerts.html) triggers audio alarm pulses. 3. Safety officer reviews alert details and clicks 'Acknowledge Alert' (POST /alerts/{id}/ack). 4. Alert transitions to Acknowledged state with recorded responder timestamp.

Workflow H: Commander Dispatches Rescue Job on Tactical Map

1. Commander opens Commander Hub (commander.html). 2. Tactical 2D map renders clustered victim pins and available rescue teams. 3. Commander clicks high-priority victim marker #VIC-01 and opens Job Dispatch modal. 4. System auto-filters eligible staff certified for Landslides; Commander assigns Team Alpha. 5. Tactical map draws animated blue route line connecting Team Alpha to Victim #VIC-01.

Workflow I: Field Responder Receives Watch Notification & Executes Rescue

1. Team Alpha member checks Watch interface (watch.html) on mobile device. 2. Screen displays high-contrast dispatch card: 'Assigned to Victim #VIC-01 · 140m NE'. 3. Responder taps 'Acknowledge Dispatch', transitioning status to EN ROUTE. 4. Upon reaching coordinates, responder taps 'Confirm Arrival', then 'Victim Rescued'. 5. Status transitions to ARRIVED / RESCUED in database and tactical map updates pin to green.

Workflow J: Analytics Logs Operational Performance Metrics

1. Operations Commander opens Analytics (analytics.html). 2. Selects 'Current Session' scope; KPI cards reflect 1 Session, 4 Detected, 4 Rescued, 100% Rescue Rate. 3. Average response time displays '1m 15s'; rescue trend chart draws complete completion bar. 4. Commander clicks 'CSV Export' to download complete mission audit log.

Workflow K: Engineering Team Reviews 3D Digital Twin & Hardware State

1. Hardware engineers open Visual 3D (visual.html). 2. Inspects physical 3D mast structures and rotates camera around solar battery stations. 3. Clicks Station #4 to verify battery voltage remaining at 12.6V after high-intensity transmission drill.

17. Frontend Data Flow & State Lifecycle Patterns

The RidgeWatch frontend strictly adheres to a unidirectional, reactive data-flow architecture across all 12 modules.

Universal Frontend Lifecycle Pattern

USER INTERACTION / TIMER PULSE ↓ CONTROLLER REQUEST ENGINE (fetchWithFallback via config.js) ↓ PARSED JSON OPERATIONAL PAYLOAD ↓ LOCAL STATE STORE UPDATE (state object) ↓ DATA INTEGRITY & VALIDATION FILTER (Zero fake data checks) ↓ DOM MUTATION & SVG / CANVAS RENDER (renderAll) ↓ VISUAL FEEDBACK & OPERATIONAL ACTION

> **[FAIL-SAFE DEGRADATION]** When network disconnection occurs, the frontend gracefully activates state.isOffline, displaying 'OFFLINE / Backend Unreachable' status badges while preserving the last valid cached telemetry in memory.

18. Comprehensive File Relationship & Dependency Matrix

The following master matrix cross-references every primary frontend file with its file type, core responsibility, primary user role, key output, and upstream dependencies.

FILE NAME	TYPE	PRIMARY RESPONSIBILITY	USER ROLE	DEPENDENCIES
index.html	HTML	Executive Operational Dashboard	Commander	style.css, config.js, dashboard.js
dashboard.js	JS	Dashboard Telemetry Poller	Commander	config.js
visual.html	HTML	2D/3D/4D Digital Twin Canvas	Engineer	style.css, config.js, visual.js
visual.js	JS	Three.js 3D Rendering Engine	Engineer	config.js, Three.js
logic.html	HTML	AI Decision Center UI	Triage Officer	style.css, config.js, logic.js
logic.js	JS	Step 0–9 AI Calculation Engine	Triage Officer	config.js
commander.html	HTML	Tactical Command Center	Commander	style.css, config.js, commander.js
commander.js	JS	2D Map Clustering & Dispatch Engine	Commander	config.js
watch.html	HTML	Field Responder Smartwatch View	Responders	style.css, config.js, watch.js
watch.js	JS	Mobile Card & Triage Controller	Responders	config.js
alerts.html	HTML	Emergency Hazard Alert Center	Safety Officer	style.css, config.js, alerts.js
alerts.js	JS	Alert Poller & Audio Alarm Engine	Safety Officer	config.js
analytics.html	HTML	Performance Analytics Dashboard	Commander	style.css, config.js, analytics.js
analytics.js	JS	Scope Metrics & SVG Chart Engine	Commander	config.js
scenarios.html	HTML	Disaster Scenario Catalog	Operator	style.css, config.js, scenarios.js
scenarios.js	JS	Scenario Launcher Controller	Operator	config.js
victims.html	HTML	Survivor Roster & Vitals View	Medical Lead	style.css, config.js, victims.js
victims.js	JS	Survivor Triage Filter Controller	Medical Lead	config.js
teams.html	HTML	Rescue Teams Directory	Logistics	style.css, config.js, teams.js
teams.js	JS	Teams Roster View Controller	Logistics	config.js
hardware.html	HTML	Hardware Telemetry Monitor	Technician	style.css, config.js, hardware.js
hardware.js	JS	Hardware Diagnostic Controller	Technician	config.js
testing.html	HTML	Simulation Testing Sandbox	Developer/QA	style.css, config.js, testing-common.js, testing.js
testing.js	JS	Simulation Stepper & Telemetry Injector	Developer/QA	config.js, testing-common.js
testing-common.js	JS	Testing Shared Nav Injector	All Testing	config.js
style.css	CSS	Master Light White/Blue Theme	All Pages	Google Fonts
config.js	JS	API Base URL Gateway	All Scripts	None

19. 6W Comprehensive Analysis Summary

Every file across the RidgeWatch frontend answers the fundamental 6W questions (What, Why, Who, Where, When, How). Detailed individual file breakdowns are provided in Section 5 (HTML) and Section 6 (JavaScript). The summary table below aggregates the overarching 6W matrix by core functional subsystem.

MODULE	WHAT?	WHY?	WHO?	WHERE?	HOW?
Dashboard	Executive overview	Immediate situational awareness	Commander	index.html	Continuous REST polling
Visual	3D Digital Twin	Spatial hardware observability	Engineers	visual.html	Three.js 60 FPS WebGL
Logical	Explainable AI	Cognitive transparency into triage	Triage Lead	logic.html	Step 0–9 mathematical proofs
Commander	Tactical theater	Coordinate multi-team dispatches	Commander	commander.html	2D spatial map clustering
Watch	Mobile responder	Glanceable field triage on wearable	Field Crew	watch.html	High-contrast card views
Alerts	Hazard ledger	Life-safety audio-visual warnings	Safety Officer	alerts.html	Web Audio API alarm pulses
Analytics	Performance audit	Evaluate rescue speed and rate	Commander	analytics.html	Scope-filtered SVG charts
Hardware	Station diagnostics	Monitor battery and packet health	Technicians	hardware.html	Live SVG gauge meters
Testing	Simulation sandbox	Validate logic without disaster	QA / Judges	testing.html	Synthetic sensor injections

20. Component Criticality & Dependency Ranking Analysis

Frontend components are classified into three strict operational tiers based on their systemic impact on mission success.

Tier 1: CRITICAL Components

• js/config.js: Single point of failure for all network communication. • css/style.css: Core visual design system and layout foundation. • commander.html & commander.js: Tactical mission command and spatial dispatch engine. • alerts.html & alerts.js: Emergency life-safety hazard notifications.

Tier 2: IMPORTANT Components

• index.html & dashboard.js: Main operational summary and quick launcher. • logic.html & logic.js: AI explainability and Bayesian fusion calculations. • analytics.html & analytics.js: Rescue performance reporting and historical audit. • watch.html & watch.js: Field responder mobile dispatch client.

Tier 3: SUPPORTING Components

• visual.html & visual.js: 3D hardware digital twin and aesthetic simulator. • hardware.html & hardware.js: Physical sensor battery and diagnostic meters. • testing*.html & testing*.js: Developer sandbox, scenario authoring, and personnel forms.

21. 'If This File Fails' Systemic Impact & Resilience Analysis

This section evaluates the fault-tolerance and operational impact if specific frontend files fail to load or encounter runtime exceptions.

FAILED FILE	FAILURE SYMPTOM	SYSTEMIC IMPACT	MITIGATION STRATEGY
config.js	API base undefined	Complete network blackout across all pages	Hardcoded relative path fallback ('')
style.css	Unstyled HTML	Broken layouts, unreadable tables	Inline critical layout fallback tokens
commander.js	Map fails to render	Cannot dispatch teams or view locations	Fallback to tabular Victims & Teams pages
alerts.js	No audio alarms	Responders unaware of new secondary hazards	Persistent visual alert banners on topbar
analytics.js	Charts unrendered	No performance trend visibility	Raw SQLite CSV export available via backend
visual.js	WebGL canvas blank	No 3D digital twin rendering	System functions normally via 2D Commander
testing.js	Sliders unresponsive	Cannot inject synthetic test readings	REST API injection via curl/Postman

22. Architectural Rationale: Why This Frontend Architecture?

The RidgeWatch frontend deliberately avoids heavy, opinionated single-page application frameworks (such as React, Angular, or Vue) in favor of modular vanilla HTML5, CSS3, and standard JavaScript ES6+.

Core Rationale Justifications

1. Zero Build-Step Dependency: Emergency systems deployed in forward operating bases or air-gapped field servers cannot depend on external npm registries, Webpack bundling, or complex Node build pipelines. RidgeWatch runs instantly from any web browser or lightweight HTTP server.

2. Fault Isolation by Module: Because each operational view operates as an independent HTML/JS document, a runtime error in Visual 3D will never crash the Incident Commander map or disable Emergency Alerts.

3. Sub-Second Cold Start Times: Vanilla HTML5 delivers instantaneous page loads with minimal memory footprint on low-power ruggedized field tablets.

23. The Complete System Frontend Story (End-to-End Narrative)

The complete operational lifecycle of a disaster response operation unfolds seamlessly across the RidgeWatch frontend in a single interconnected narrative:

1. SYSTEM INITIALIZATION & CONFIGURATION: Admin registers certified search-and-rescue personnel (testing-add-person.html) and forms specialized rescue units (testing-add-team.html). 2. DISASTER STRIKES & SCENARIO ACTIVATION: A major landslide occurs in the Himalayan sector. Commander launches the scenario from the Dashboard (index.html). 3. REAL-TIME SENSING & TELEMETRY STREAMING: Inclinometers and rain gauges stream live readings. Hardware dashboard (hardware.html) and Visual 3D (visual.html) track telemetry pulses. 4. AI COGNITIVE DECISION & FUSION: Logical AI Decision Center (logic.html) fuses multi-modal evidence across Steps 0–9, deriving a 88.4% Survival Probability Score. 5. EMERGENCY HAZARD ALERT DISPATCH: Secondary landslide risk triggers a CRITICAL network alert. Alerts module (alerts.html) sounds audible warning pulses. 6. TACTICAL COMMAND & MAP DISPATCH: Incident Commander (commander.html) inspects the clustered 2D tactical map and dispatches Team Alpha to Survivor #VIC-01. 7. FIELD RESCUE EXECUTION: Team Alpha responder checks mobile Watch client (watch.html), navigates to target coordinates, and confirms arrival. 8. OPERATIONAL AUDIT & PERFORMANCE REVIEW: Operations Commander reviews Analytics (analytics.html), verifying a 100% rescue success rate and 1m 15s average response speed.

24. Implementation Status, Compatibility Aliases & Partial Areas

In strict compliance with factual accuracy, this section identifies fully implemented operational modules versus compatibility aliases and partial features.

FRONTEND COMPONENT	IMPLEMENTATION STATUS	TECHNICAL ASSESSMENT
index.html / dashboard.js	FULLY IMPLEMENTED	Complete polling, KPI rendering, mini-map, quick scenario trigger
visual.html / visual.js	FULLY IMPLEMENTED	Complete Three.js 3D digital twin, 2D schematic, 4D simulation
logic.html / logic.js	FULLY IMPLEMENTED	Complete Level 1–4 AI decision center, Step 0–9 mathematical proofs
commander.html / commander.js	FULLY IMPLEMENTED	Complete 2D map clustering, collision avoidance, job assignment drawer
watch.html / watch.js	FULLY IMPLEMENTED	Complete mobile responder view, vital signs card, acknowledgment button
alerts.html / alerts.js	FULLY IMPLEMENTED	Complete active alerts ledger, severity filtering, Web Audio alarm
analytics.html / analytics.js	FULLY IMPLEMENTED	Complete global scope filtering (Live to All-Time), SVG trend charts
hardware.html / hardware.js	FULLY IMPLEMENTED	Complete station health grid, battery meters, diagnostic telemetry
testing.html / testing*.js	FULLY IMPLEMENTED	Complete simulation sandbox, step controls, synthetic telemetry sliders
admin.html & admin-*.html	COMPATIBILITY ALIAS	Client-side redirect gateways routing legacy URLs to testing-*.html
admin.js & admin-*.js	COMPATIBILITY PROXY	Lightweight JavaScript wrappers delegating to testing-*.js controllers

25. Frontend Architecture Summary & Final Conclusions

The RIDGEWATCH frontend architecture delivers a robust, highly modular, and mission-critical emergency rescue interface. By unifying real-time telemetry streaming, explainable AI cognitive decision modeling, dynamic 2D tactical map clustering, and immersive 3D digital twin visualization, it equips disaster response teams with the tools needed to save lives in the critical golden hour.

Summary of Frontend Achievements

• **54 Verified Frontend Files:** Every HTML view, JavaScript controller, and CSS rule has been audited, verified, and conceptually documented.

• **Zero Fake Data Mandate:** Strict mathematical relationships guarantee that all operational reporting reflects audited backend simulation states.

• **Production Readiness:** Validated across full automated regression test suites (565 passing tests) and confirmed ready for mission evaluation, hackathon judging, and field deployment.

> **[END OF TECHNICAL SPECIFICATION]** RIDGEWATCH — AI-Based Multi-Disaster Survivor Detection & Rescue Coordination System. Frontend Architecture & Conceptual Documentation v2.0.